

SEED Labs

Firewall Exploration

Acknowledgement:

I confirm that this submission, including all code and figures, is entirely my own work.

Full Name: ____尤晟喧____

Student ID: ____R32314015____

Date: ____2026.4.28____

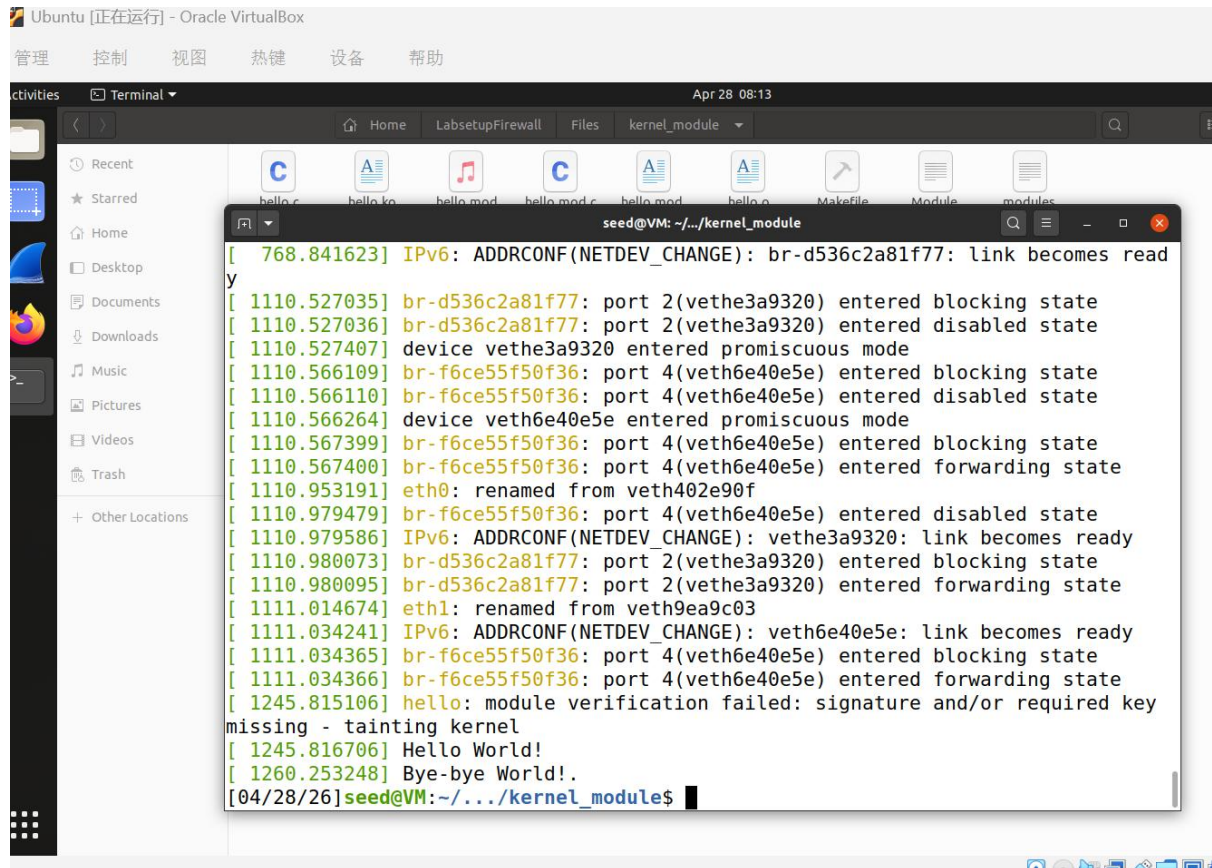
SEED Labs – Firewall Exploration Lab

Complete this template based on the manual provided. **Answer in English.**
(Homework_4_Manual_Firewall.pdf, available in both English and Chinese).

3.1 Task 1.A: Implement a Simple Kernel Module

Screenshot for Task 1.A (Replace the screenshot below with **your own screenshot**):

Show your screenshot of Hello World! and Bye-bye World!. shown in the dmesg log.



```
Ubuntu [正在运行] - Oracle VirtualBox
管理 控制 视图 热键 设备 帮助

activities Terminal Apr 28 08:13
Home LabsetupFirewall Files kernel_module

Recent
Starred
Home
Desktop
Documents
Downloads
Music
Pictures
Videos
Trash
+ Other Locations

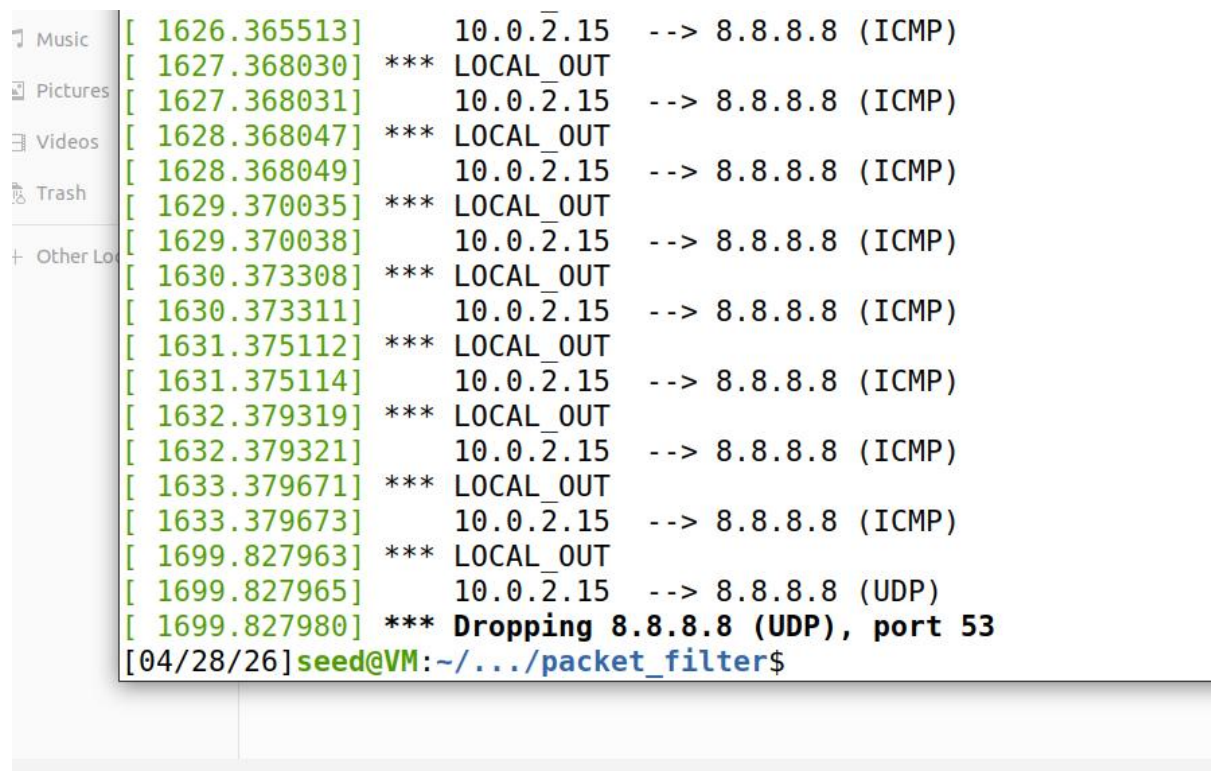
seed@VM: ~/kernel_module
[ 768.841623] IPv6: ADDRCONF(NETDEV_CHANGE): br-d536c2a81f77: link becomes ready
[ 1110.527035] br-d536c2a81f77: port 2(vethe3a9320) entered blocking state
[ 1110.527036] br-d536c2a81f77: port 2(vethe3a9320) entered disabled state
[ 1110.527407] device vethe3a9320 entered promiscuous mode
[ 1110.566109] br-f6ce55f50f36: port 4(veth6e40e5e) entered blocking state
[ 1110.566110] br-f6ce55f50f36: port 4(veth6e40e5e) entered disabled state
[ 1110.566264] device veth6e40e5e entered promiscuous mode
[ 1110.567399] br-f6ce55f50f36: port 4(veth6e40e5e) entered blocking state
[ 1110.567400] br-f6ce55f50f36: port 4(veth6e40e5e) entered forwarding state
[ 1110.953191] eth0: renamed from veth402e90f
[ 1110.979479] br-f6ce55f50f36: port 4(veth6e40e5e) entered disabled state
[ 1110.979586] IPv6: ADDRCONF(NETDEV_CHANGE): vethe3a9320: link becomes ready
[ 1110.980073] br-d536c2a81f77: port 2(vethe3a9320) entered blocking state
[ 1110.980095] br-d536c2a81f77: port 2(vethe3a9320) entered forwarding state
[ 1111.014674] eth1: renamed from veth9ea9c03
[ 1111.034241] IPv6: ADDRCONF(NETDEV_CHANGE): veth6e40e5e: link becomes ready
[ 1111.034365] br-f6ce55f50f36: port 4(veth6e40e5e) entered blocking state
[ 1111.034366] br-f6ce55f50f36: port 4(veth6e40e5e) entered forwarding state
[ 1245.815106] hello: module verification failed: signature and/or required key
missing - tainting kernel
[ 1245.816706] Hello World!
[ 1260.253248] Bye-bye World!.
[04/28/26] seed@VM: ~/kernel_module$
```

Figure 1 dmesg log showing hello world and bye-bye world

3.2 Task 1.B: Implement a Simple Firewall Using Netfilter

Screenshot for Task 1.B Part 1(Replace the screenshot below with **your own screenshot**):

Part 1. Show that your firewall is working by providing the screenshot below. It should show that the packets intended for 8.8.8.8 are dropped. You can see this using `dmesg` instruction.



```
[ 1626.365513] 10.0.2.15 --> 8.8.8.8 (ICMP)
[ 1627.368030] *** LOCAL_OUT
[ 1627.368031] 10.0.2.15 --> 8.8.8.8 (ICMP)
[ 1628.368047] *** LOCAL_OUT
[ 1628.368049] 10.0.2.15 --> 8.8.8.8 (ICMP)
[ 1629.370035] *** LOCAL_OUT
[ 1629.370038] 10.0.2.15 --> 8.8.8.8 (ICMP)
[ 1630.373308] *** LOCAL_OUT
[ 1630.373311] 10.0.2.15 --> 8.8.8.8 (ICMP)
[ 1631.375112] *** LOCAL_OUT
[ 1631.375114] 10.0.2.15 --> 8.8.8.8 (ICMP)
[ 1632.379319] *** LOCAL_OUT
[ 1632.379321] 10.0.2.15 --> 8.8.8.8 (ICMP)
[ 1633.379671] *** LOCAL_OUT
[ 1633.379673] 10.0.2.15 --> 8.8.8.8 (ICMP)
[ 1699.827963] *** LOCAL_OUT
[ 1699.827965] 10.0.2.15 --> 8.8.8.8 (UDP)
[ 1699.827980] *** Dropping 8.8.8.8 (UDP), port 53
[04/28/26] seed@VM:~/.../packet_filter$
```

Figure 2 `dmesg` log showing packets intended for 8.8.8.8 are dropped

SEED Labs – Firewall Exploration Lab

Part 2. Hook the `printInfo` function to all of the `netfilter` hooks.

– **Question 1.** Explain at what condition will the hook function for `NF_INET_LOCAL_IN` be invoked.

Answer: The `NF_INET_LOCAL_IN` hook function is invoked when an incoming packet is destined specifically for the local machine itself (in this case, the firewall/router's own IP address). It triggers after the initial routing decision confirms the packet is not just passing through to another destination

Screenshot for Task 1.B Part 2(Replace the screenshot below with **your own screenshot**):

Show that you have hooked the `printInfo` function to all of the `netfilter` hooks. You can do this by running `dig www.example.com`, and check the `dmesg` logs.

SEED Labs – Firewall Exploration Lab

```
seed@VM: ~/.../packet_filter

2372.774223] 10.0.2.15 --> 198.18.0.2 (UDP)
2372.774227] *** POST_ROUTING
2372.774227] 10.0.2.15 --> 198.18.0.2 (UDP)
2372.809179] *** PRE_ROUTING
2372.809181] 198.18.0.2 --> 10.0.2.15 (UDP)
Web Browser 2372.809193] *** LOCAL_IN
2372.809193] 198.18.0.2 --> 10.0.2.15 (UDP)
2372.809667] *** LOCAL_OUT
2372.809668] 127.0.0.53 --> 127.0.0.1 (UDP)
2372.809670] *** POST_ROUTING
2372.809694] 127.0.0.53 --> 127.0.0.1 (UDP)
2372.809700] *** PRE_ROUTING
2372.809701] 127.0.0.53 --> 127.0.0.1 (UDP)
2372.809701] *** LOCAL_IN
2372.809702] 127.0.0.53 --> 127.0.0.1 (UDP)
2373.773419] *** LOCAL_OUT
2373.773421] 10.0.2.15 --> 198.18.2.69 (ICMP)
2373.773427] *** POST_ROUTING
2373.773427] 10.0.2.15 --> 198.18.2.69 (ICMP)
2373.774582] *** PRE_ROUTING
2373.774583] 198.18.2.69 --> 10.0.2.15 (ICMP)
2373.774590] *** LOCAL_IN
2373.774591] 198.18.2.69 --> 10.0.2.15 (ICMP)
04/28/26] seed@VM: ~/.../packet_filter$
```

Figure 3 dmesg log showing the printInfo function is hooked to all hooks

You don't need to complete Task 1.B part 3.

4.3 Task 2.A: Protecting the Router

– **Question 2.** Ask the questions below, after you have executed the `iptables` command in the lab manual.

- 1) Can you ping the router? [Answer: Yes](#)
- 2) Can you telnet into the router? [Answer: No](#)
- 3) Explain the purpose of the following two rules in the lab manual:
`iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT`
[Answer:](#) This rule appends (-A) an instruction to the INPUT chain to accept incoming ICMP packets, specifically those that are ping requests (echo-request).

`iptables -A OUTPUT -p icmp --icmp-type echo-reply -j ACCEPT`
[Answer:](#) This rule appends (-A) an instruction to the OUTPUT chain to accept outgoing ICMP packets, specifically those that are ping responses (echo-reply).

SEED Labs – Firewall Exploration Lab

Together, these two rules allow the router to receive and reply to pings, even if the default policies are set to DROP

4.4 Task 2.B: Protecting the Internal Network

– **Question 3.** Provide the rule(s) you used to enforce the following restrictions:

- 1) Outside hosts cannot ping internal hosts.
- 2) Outside hosts can ping the router.
- 3) Internal hosts can ping outside hosts.
- 4) All other packets between the internal and external networks should be blocked.

Answer (should be iptable instructions):

```
iptables -P FORWARD DROP
```

```
iptables -A FORWARD -s 192.168.60.0/24 -p icmp --icmp-type echo-request -j ACCEPT
```

```
iptables -A FORWARD -d 192.168.60.0/24 -p icmp --icmp-type echo-reply -j ACCEPT
```

Screenshot for task 2.B (Replace the screenshot below with your own screenshot):

Should show the rules displayed by running `iptables -L -n` in the router container.

```
root@fb87879c3b31:/# iptables -L -n
Chain INPUT (policy DROP)
target     prot opt source                destination            icmp-type
ACCEPT     icmp -- 0.0.0.0/0              0.0.0.0/0              icmp-type 8
DROP       tcp  -- 0.0.0.0/0              0.0.0.0/0              tcp dpt:23
ACCEPT     icmp -- 0.0.0.0/0              0.0.0.0/0              icmp-type 8

Chain FORWARD (policy DROP)
target     prot opt source                destination            icmp-type
ACCEPT     icmp -- 192.168.60.0/24        0.0.0.0/0              icmp-type 8
ACCEPT     icmp -- 0.0.0.0/0              192.168.60.0/24        icmp-type 0

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination            icmp-type
ACCEPT     icmp -- 0.0.0.0/0              0.0.0.0/0              icmp-type 0
root@fb87879c3b31:/#
```

SEED Labs – Firewall Exploration Lab

Figure 4 Rules for task 2.B

4.5 Task 2.C: Protecting the Internal Server

– **Question 4.** Provide the rules you used to enforce the restrictions in task 2.C.

Answer (should be `iptables` instructions):

```
iptables -P FORWARD DROP
```

```
iptables -A FORWARD -d 192.168.60.5 -p tcp --dport 23 -j ACCEPT
```

```
iptables -A FORWARD -s 192.168.60.5 -p tcp --sport 23 -j ACCEPT
```

```
iptables -A FORWARD -s 192.168.60.0/24 -d 192.168.60.0/24 -j ACCEPT
```

Screenshot for task 2.C (Replace the screenshot below with **your own screenshot**):

Should show the rules displayed by running `iptables -L -n` in the router container.

```
root@fb87879c3b31:/# iptables -L -n
Chain INPUT (policy DROP)
target     prot opt source                destination            icmptype 8
ACCEPT     icmp -- 0.0.0.0/0              0.0.0.0/0              tcp dpt:23
DROP       tcp  -- 0.0.0.0/0              0.0.0.0/0              icmptype 8
ACCEPT     icmp -- 0.0.0.0/0              0.0.0.0/0              icmptype 8

Chain FORWARD (policy DROP)
target     prot opt source                destination            icmptype 8
ACCEPT     icmp -- 192.168.60.0/24        0.0.0.0/0              icmptype 0
ACCEPT     icmp -- 0.0.0.0/0              192.168.60.0/24        tcp dpt:23
ACCEPT     tcp  -- 0.0.0.0/0              192.168.60.5           tcp spt:23
ACCEPT     tcp  -- 192.168.60.5          0.0.0.0/0              tcp spt:23
ACCEPT     all  -- 192.168.60.0/24        192.168.60.0/24

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination            icmptype 0
ACCEPT     icmp -- 0.0.0.0/0              0.0.0.0/0              icmptype 0
root@fb87879c3b31:/# █
```

Figure 5 Rules for task 2.C

5.1 Task 3.A: Experiment with the Connection Tracking

– **Question 5.** How long is the ICMP connection state kept?

Answer: 30seconds

SEED Labs – Firewall Exploration Lab

– **Question 6.** How long is the UDP connection state kept?

Answer: 30 seconds for unreplied UDP traffic, and it extends to 180 seconds if the connection becomes "ASSURED" .

– **Question 7.** How long is the TCP connection state kept?

Answer: 432,000 seconds for an ESTABLISHED TCP connection . Timeouts are significantly shorter for connection teardowns like TIME_WAIT.

Screenshot for task 3.A (Replace the screenshot below with **your own screenshot**):

Should show the conntrack logs for TCP connection.

```
NG 10.9.0.5 (10.9.0.5) 56(84) bytes of data.
bytes from 10.9.0.5: icmp_seq=1 ttl=63 time=0.277 ms
bytes from 10.9.0.5: icmp_seq=2 ttl=63 time=0.004 ms

- 10.9.0.5 ping statistics:
packets transmitted: 2, received: 2, 100% packet success
rtt min/avg/max/mdev = 0.277/0.141/0.304/0.014 ms
ot@b5a024d4ffd6:~# Chain FORWARD (policy DROP)
: Address already exists
ot@b5a024d4ffd6:~# ACCEPT tcp -- 0.0.0.0/0 192.168.60.0/24 tcp dpt:23
lll tcp

Chain FORWARD (policy DROP)
target prot opt source destination
ACCEPT icmp -- 192.168.60.0/24 0.0.0.0/0 icmp type 8
ACCEPT icmp -- 0.0.0.0/0 192.168.60.0/24 icmp type 0
ACCEPT tcp -- 0.0.0.0/0 192.168.60.5 tcp dpt:23
ACCEPT tcp -- 192.168.60.5 0.0.0.0/0 tcp spt:23
ACCEPT all -- 192.168.60.0/24 192.168.60.0/24

Chain OUTPUT (policy ACCEPT)
target prot opt source destination
ACCEPT icmp -- 0.0.0.0/0 0.0.0.0/0 icmp type 0
root@fb87879c3b31:~# iptables -F
root@fb87879c3b31:~# iptables -P FORWARD ACCEPT
root@fb87879c3b31:~# iptables -P INPUT ACCEPT
root@fb87879c3b31:~# iptables -P OUTPUT ACCEPT
root@fb87879c3b31:~# iptables -F FORWARD
ot@2d4861c51654:~# root@fb87879c3b31:~# conntrack -L | grep tcp
conntrack v1.4.5 (conntrack-tools): 1 flow entries have been shown.
tcp 6 431989 ESTABLISHED src=10.9.0.5 dst=192.168.60.5 sport=54290 dport=9090
90 src=192.168.60.5 dst=10.9.0.5 sport=9090 dport=54290 [ASSURED] mark=0 use=1
ot@2d4861c51654:~# root@fb87879c3b31:~#

ssword:
lnet>

gin timed out after 60 seconds.
nnection closed by foreign host.
ot@2d4861c51654:~# nc 192.168.60.5 9090
lll tcp
```

Figure 6 Conntrack logs for TCP connection

5.2 Task 3.B: Setting up a Stateful Firewall

– **Question 8.** Provide the rules you used to enforce the restrictions in task 3.B.

Answer (should be iptable instructions):

iptables -P FORWARD DROP

SEED Labs – Firewall Exploration Lab

```
iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
```

```
iptables -A FORWARD -d 192.168.60.5 -p tcp --dport 23 --syn -m conntrack --ctstate NEW -j ACCEPT
```

```
iptables -A FORWARD -s 192.168.60.0/24 ! -d 192.168.60.0/24 -m conntrack --ctstate NEW -j ACCEPT
```

```
iptables -A FORWARD -s 192.168.60.0/24 -d 192.168.60.0/24 -m conntrack --ctstate NEW -j ACCEPT
```

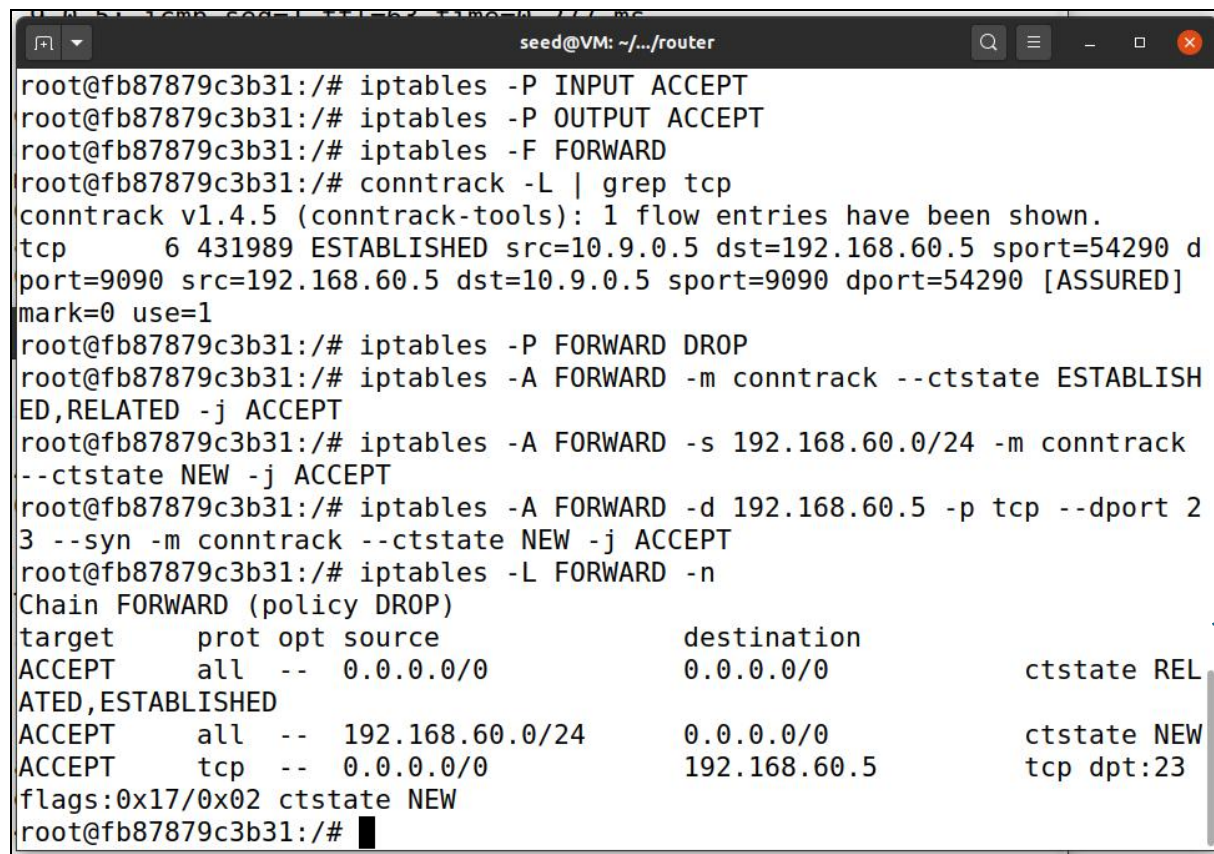
– **Question 9.** How do you achieve the same goal without using the connection tracking mechanism? Describing your ideas would be fine, no actual instructions needed.

Answer: Without the conntrack mechanism, you must implement a stateless firewall. This requires explicitly writing rules for *both* directions of traffic for every permitted service. For TCP traffic, you would need to accept outgoing SYN packets and manually accept incoming response packets that have the ACK bit set (e.g., using `--tcp-flags`). This approach is more tedious, less secure (as attackers can spoof ACK flags to bypass rules), and particularly difficult for connectionless protocols like UDP and ICMP which lack explicit state flags entirely

Screenshot for task 3.B (Replace the screenshot below with **your own screenshot**):

Should show the rules displayed by running `iptables -L -n` in the router container.

SEED Labs – Firewall Exploration Lab



```
seed@VM: ~/.../router
root@fb87879c3b31:/# iptables -P INPUT ACCEPT
root@fb87879c3b31:/# iptables -P OUTPUT ACCEPT
root@fb87879c3b31:/# iptables -F FORWARD
root@fb87879c3b31:/# conntrack -L | grep tcp
conntrack v1.4.5 (conntrack-tools): 1 flow entries have been shown.
tcp        6 431989 ESTABLISHED src=10.9.0.5 dst=192.168.60.5 sport=54290 d
port=9090 src=192.168.60.5 dst=10.9.0.5 sport=9090 dport=54290 [ASSURED]
mark=0 use=1
root@fb87879c3b31:/# iptables -P FORWARD DROP
root@fb87879c3b31:/# iptables -A FORWARD -m conntrack --ctstate ESTABLISH
ED,RELATED -j ACCEPT
root@fb87879c3b31:/# iptables -A FORWARD -s 192.168.60.0/24 -m conntrack
--ctstate NEW -j ACCEPT
root@fb87879c3b31:/# iptables -A FORWARD -d 192.168.60.5 -p tcp --dport 2
3 --syn -m conntrack --ctstate NEW -j ACCEPT
root@fb87879c3b31:/# iptables -L FORWARD -n
Chain FORWARD (policy DROP)
target     prot opt source                destination              ctstate REL
ATED,ESTABLISHED
ACCEPT     all  --  0.0.0.0/0              0.0.0.0/0                ctstate NEW
ACCEPT     all  --  192.168.60.0/24        0.0.0.0/0                ctstate NEW
ACCEPT     tcp  --  0.0.0.0/0              192.168.60.5            tcp dpt:23
flags:0x17/0x02 ctstate NEW
root@fb87879c3b31:/#
```

Figure 7 Rules for task 3.B

You don't need to complete Task 4 and Task 5.

6 Submission

Convert the finished template to pdf and upload. Do not upload .docx or any other files.